07

Secrets and configuration

Where keys live, and which of them must never reach the browser

An app depends on values that don't belong in its code: settings that differ between environments, and credentials that grant access to databases, payment accounts, and email senders. Both kinds live in configuration, and the failures on this surface share a property that makes them worth a chapter: they're one-directional. A secret that reaches a browser is public, and stays public after it's removed. This chapter covers where these values live, how to tell the two kinds of key apart, and what to do when one has been where it shouldn't.

Assumes chapter 1's two sides and chapter 5's environments.

PART

The model

Configuration against code

Code is the same in every environment; configuration is what differs: which database to talk to, which features are on, which keys to use. Keeping the two separate is what lets one build run everywhere — and it's also the security boundary, since configuration is where credentials live and code is what gets committed, shared, and shipped.

The mechanism is the environment variable (chapter 1): locally they're read from a `.env` file, and in production from the platform's dashboard. The repository carries an example file listing which variables exist, with the values blank — real values never get committed, because a repository is a shared artifact with a permanent memory.

Two kinds of key

Platforms issue keys in pairs, and the two kinds have opposite rules.

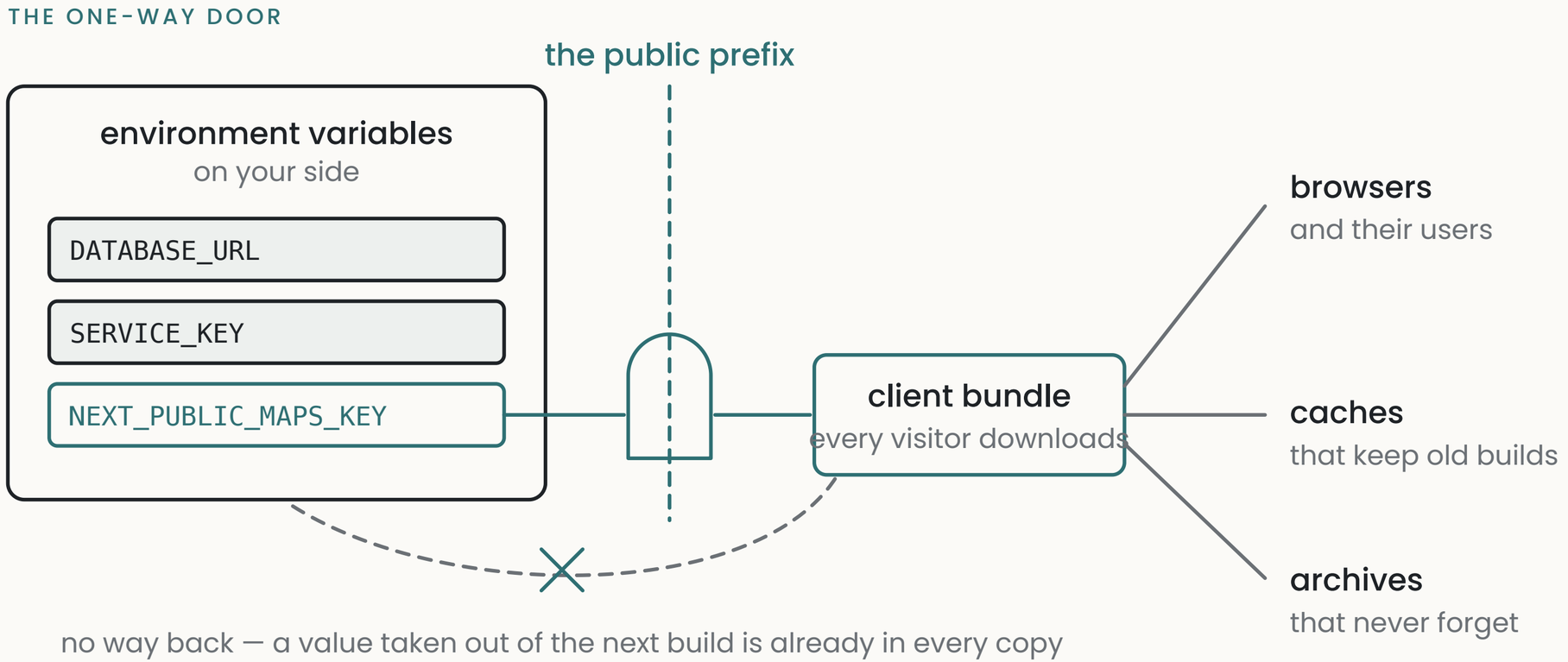
A **public key** is designed to ship in the browser. It identifies your app rather than trusting the holder, and it's safe in the open only because the real rules are enforced elsewhere — the database policies of chapter 6. Finding it in devtools is expected.

A **privileged key** bypasses those rules; it exists for trusted server-side work, and it must never reach a browser, a shared file, or a log. Platforms name the pair so you can tell them apart — `anon` against `service_role`, publishable against secret — and the naming is the first thing to check when a key appears somewhere new. Other platforms name the pair differently — a web key against an admin credential, publishable against secret — and the rule is the same. A privileged key that can spend turns exposure into a bill, which is often how a leak is first noticed.

The one-way door

Client code is bundled: compiled into files that every visitor downloads. Frameworks mark which environment variables are allowed into the bundle with a naming prefix — `NEXT_PUBLIC_`, `VITE_`, and equivalents — and the prefix is a one-way door: any value carrying it is copied into files the whole internet receives. A privileged key that was ever bundled has been distributed; taking it out of the next build doesn't recall the copies already in browsers, caches, and archives. The same permanence applies to a secret committed to a repository — removing it from the current version leaves it in the history, which anyone with the repository can read.

So exposure is answered by rotation rather than deletion, since deletion doesn't reach the copies already made.



Rotation

Rotating a key means issuing a new one, updating every legitimate place the old one was used, and revoking the old one. It's a routine operation on every platform, and worth doing once calmly — the alternative is learning the steps for the first time during an incident. A key that was ever exposed gets rotated on that fact alone; whether anyone found it is a separate question (your logs may answer it), and waiting to find out extends the exposure.

What counts as exposed

A working list: committed to a repository, including in its history; carried in the client bundle; pasted into an issue, a chat message, or a screenshot; printed into logs. Each is a place with its own audience and its own memory, and most of them can't be un-shared.

PART

What goes wrong

1 A secret reaching the client

a private value carried through the one-way door.

ASK "List every environment variable, whether it's public or private, and which of them the browser receives. Flag any private one with a public prefix."

CHECK search the built client bundle for each secret's value.

TELL *a key with a public-variable prefix that isn't a public key, or a secret referenced in a file the bundler includes.*

2 The privileged key doing ordinary work

the bypass credential used where the public key belongs.

ASK "List every place the privileged key is used, and for each, why the public key can't do that work."

CHECK the list should be short, server-only, and each entry should survive the why question.

TELL *the privileged key in ordinary read or write paths, or a commit that fixes a permissions error without touching a policy (chapter 6's fourth shape, seen from this side).*

3 A live key in a shared artifact

a credential in something with an audience and a memory.

ASK “Search the repository and its history for values that look like live credentials. List the file and the kind of key — don’t print the values.”

CHECK any hit in history means rotate, since removing the value from the current version leaves it in every earlier one.

TELL *real-looking values in files about to be committed, in issues, in pasted logs.*

THE DIRECT TEST

Build the client and search the output for the first characters of each secret — chapter 0’s D3 prompt does both and shows you the command. It takes minutes and converts “the bundler shouldn’t include it” into an observation about what the bundler did. Repeat it after dependency or framework changes — bundling behaviour is configuration too, and it drifts.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · The credential inventory

AUDIT

List every credential my app uses: what it grants access to, whether it's public-by-design or privileged, where it's stored for each environment, whether the browser can ever receive it, and when it was last rotated. Full table; write unknown where the answer is unknown.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response has a row per key and honest unknowns — "last rotated: unknown" is the normal state of a young project and the start of the fix. Follow up on any privileged key whose browser column isn't a plain no.

D2 · Explain my keys to me

GROUNDED EXPLANATION

Using my own project: show me where each of my keys is used, and for each one, what someone who obtained it could do — which tables they could read or write, what they could spend, what they could send. Distinguish what the key itself allows from what my policies would still prevent.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response makes the public/privileged distinction concrete with your own tables and accounts. The last sentence of each entry is the one that matters: what the policies would still prevent is exactly chapter 6's subject.

D3 · Walk the leak

WALK THE FAILURE PATH

Suppose my privileged key leaked an hour ago. Walk me through it: what could have been done with it already, how I'd find out from my logs whether anything was, and then the rotation itself for my exact setup — each step, what breaks during it, and how long the whole thing takes.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is a procedure you could execute today. Keep it — it's the incident runbook, written on a calm day.

D4 · The bundle search

BUILD A TOY

Help me run the direct test: build my client the way production builds it, then search the output for the first eight characters of each of my secrets, then search my repository's full history for the same. Show me the exact commands and help me read the results.

WHAT A GOOD RESPONSE LOOKS LIKE

It takes about fifteen minutes. Two empty searches are the pass; either one non-empty means a rotation, since removing the value doesn't recall the copies.

WHERE THIS CONNECTS

Chapter 1 · Client and server explains why the browser's copy is public. **Chapter 6 · Authorization** is the other half of the public key's safety — the policies that make it safe to ship. **Chapter 5 · Production** owns the environments this configuration varies across.